DICE-SASPy: Convergence-Controlled Agents for Repository-Level SAS-to-Python Migration

Anonymous Authors[1]

Abstract

Repository-level code migration remains difficult for current LLM systems because finite context windows, dense dependencies, schema lineage, and output divergence require executable validation. We model legacy data science pipeline migration as controlled search: an LLM proposes local translations, while a deterministic controller accepts edits only when matched-input validation preserves source-target equivalence or does not increase divergence. We present **DICE** (Dynamic Inhomogeneous Conversion Engine), a language-agnostic controller for convergence-controlled migration, and instantiate it as **DICE-SASPy** for SAS-to-Python migration. **DICE**'s inhomogeneous property means that partially converted source and target code can run together, allowing mixed SAS/Python validation before full conversion. The framework coordinates repository analysis, dynamic chunking, validation, rollback, and human escalation. The contribution is an inspectable migration-control architecture and a protocol for future empirical evaluation.

1. Introduction

Large language models and coding agents now handle many localized programming tasks well enough that the harder remaining challenge is repository-level migration, not isolated synthesis. That challenge is also unlikely to become a one-shot problem even as models improve. Finite context windows expose only part of the repository, cross-file dependencies are hard to preserve, and correctness often requires executable, environment-aware validation (Tao et al., 2025; Pan et al., 2026). Project-scale translation systems and benchmarks show why decomposition and project-aware validation matter under real dependencies and whole-project constraints (Ibrahimzada et al., 2024; Zhang et al., 2025; Wang et al., 2024). This is why repository-scale reasoning is increasingly pushed into agentic workflows through decomposition, retrieval, tool use, and iterative repair. The missing ingredient is therefore not another prompt tweak, but a mechanism for decomposition, validation, retries, and rollback.

Recent work suggests the form such a mechanism should take. Execution feedback, tool-using repair, and reinforcement from runtime signals provide useful models for iterative correction (Gehring et al., 2024; Bouzenia et al., 2024). For translation, runtime state alignment and fine-grained traces help localize semantic errors beyond final I/O mismatches alone (Li et al., 2025; Yuan et al., 2026). Together, these strands motivate migration systems that are not merely *feedback-aware*, but *feedback-controlled*.

Data science and analytics pipelines are a natural testbed because they expose schemas, intermediate tables, model artifacts, and downstream metrics as validation boundaries. Legacy SAS analytics in banking, insurance, healthcare, and pharmaceuticals are a particularly sharp instance because these pipelines sit on exactly these validation surfaces and face strong regulatory and mission-critical pressure to modernize. Industry guidance suggests that LLM coding agents can help analyze legacy systems and recover undocumented dependencies, but mission-critical migrations still require rigorous executable validation (Anthropic, 2026). The SAS→Python setting sharpens this tension: SAS is a legacy, comparatively low-resource language for modern LLM tooling, and SAS repositories encode business and scientific logic through macros, implicit typing, formats, and table lineage; SASPy (SAS Institute Inc., 2024) makes paired output checks feasible when translations could otherwise drift in row counts, missing-value behavior, key integrity, summaries, or downstream metrics. This mixed SAS/Python execution model is the inhomogeneous mechanism in DICE: partially converted modules can be validated together while the repository remains in transition.

In this setting, **convergence-controlled** means that each accepted proposal must preserve source-target output equivalence or reduce measured output divergence under deterministic validation. The LLM is not treated as an au-

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

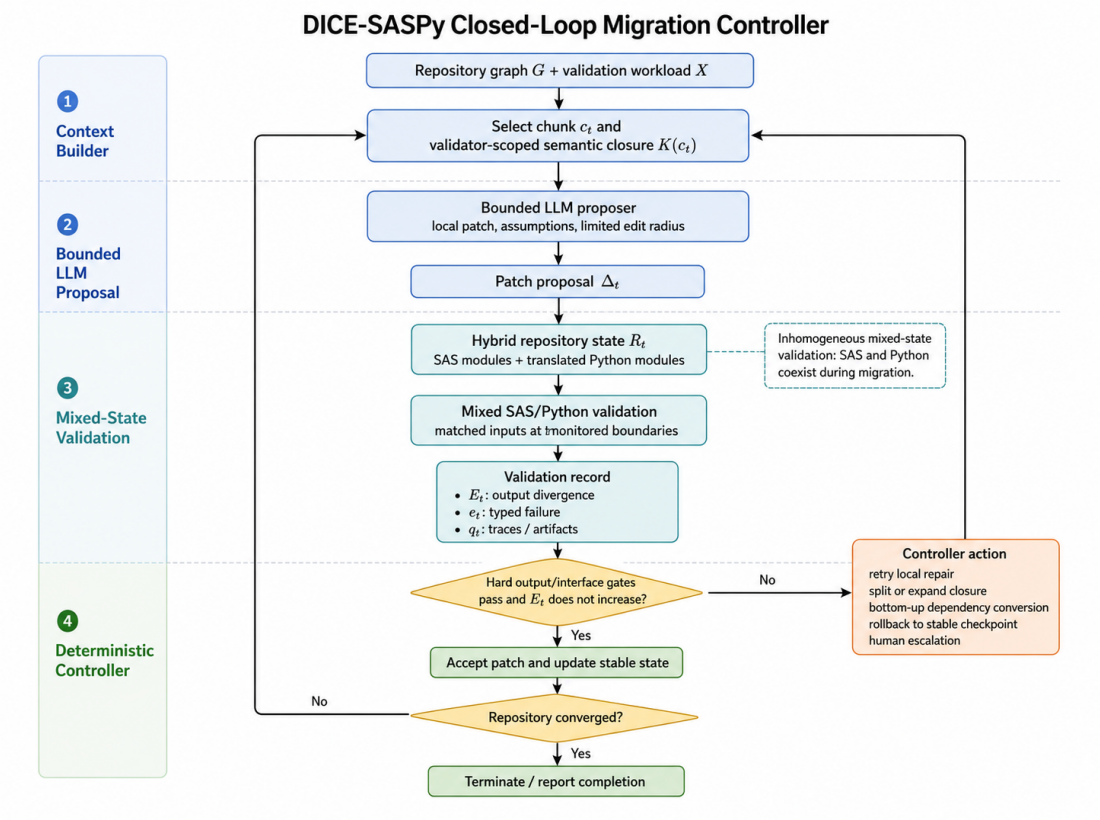

Figure 1. Closed-loop DICE controller, instantiated here by DICE-SASPy for SAS/Python migration.

tonomous translator, but as a stochastic proposer operating under controller-imposed bounds on context, chunk size, interface contracts, and retry budget. Validation failures become control signals that determine the next controller action: re-prompting, smaller decomposition, expansion of the semantic closure, rollback to a stable checkpoint, bottom-up conversion of supporting leaves, or escalation to a human reviewer. The controller therefore measures *progress* toward output equivalence, rather than only final *success*.

This paper makes three scoped contributions. First, it formulates repository-level migration as a language-agnostic, convergence-controlled search problem over a repository graph, with explicit semantic closure, validation outputs, controller actions, and an operational convergence criterion. Second, it introduces **DICE** as a modular controller architecture combining semantic closure, dynamic chunking, inhomogeneous mixed-state runtime validation, rollback, and human oversight, and realizes that design in **DICE-SASPy** for SAS→Python migration. Third, it specifies a compact future evaluation plan for DICE-SASPy covering baseline families, core metrics, and repository-availability constraints. The paper is intentionally design-forward: it argues for an inspectable controller architecture and defines how that architecture should be evaluated, rather than reporting completed migration experiments.

2. Related Work

DICE is related to four lines of work: repository-level code translation, repository context and executable environment alignment, execution-guided repair, and legacy analytics modernization.

Repository-level code translation. Early studies evaluate isolated functions or programming-contest tasks; recent work scales to repository-level translation where correctness depends on cross-file dependencies, build setup, tests, and language-specific idioms. AlphaTrans, RepoTransBench, and Zhang et al. demonstrate both the approach and its remaining difficulty (Ibrahimzada et al., 2024; Wang et al., 2024; Zhang et al., 2025). These systems motivate DICE's decomposition and validation design, but DICE differs in control: rather than only translating fragments or validating completed repositories, it treats the migration trajectory as the controlled process, where every accepted change must preserve or improve measured source-target output agreement.

Repository context and executable environment alignment. Repository-level tasks require recovering context

from large, noisy codebases. Retrieval-augmented generation, context-compression, and environment-alignment work identify long-range dependencies and executable setup as central constraints (Tao et al., 2025; Feng et al., 2026; Pan et al., 2026). DICE specializes this for migration: the relevant context is the validator-scoped semantic closure of the current chunk, including dependencies, schemas, lineage, interfaces, and downstream consumers.

Execution feedback, repair, and coding agents. Runtime feedback improves generated or repaired code. RLEF, RepairAgent, SWE-agent, and TransAgent demonstrate execution-guided iteration for synthesis, repair, and trace alignment (Gehring et al., 2024; Bouzenia et al., 2024; Yang et al., 2024; Yuan et al., 2026), but their feedback loops typically address bugs, tests, or traces. DICE instead turns validation feedback into explicit migration-control actions: accept, retry, split, roll back, convert dependencies bottom-up, or escalate.

Legacy analytics and SAS-to-Python modernization. Industry guidance shows agentic tools help analyze legacy systems, recover dependencies, and preserve business logic during migration (Anthropic, 2026). Legacy analytical pipelines are particularly demanding because correctness is expressed through tables, schemas, missing-value behavior, summaries, reports, and downstream metrics rather than through unit-test assertions alone. SAS pipelines sharpen the problem further, since SAS programs combine macros, DATA steps, PROC calls, implicit typing, formats, include files, and table lineage in ways that resist purely syntactic translation. SASPy enables mixed Python/SAS execution and monitored output comparisons (SAS Institute Inc., 2024), and DICE-SASPy uses that bridge for runtime validation rather than only interoperability.

Positioning. In summary, prior work provides important mechanisms for decomposition, retrieval, executable validation, trace alignment, and tool-using repair. DICE combines these ideas under a different abstraction: convergence-controlled migration. The LLM is treated as a bounded stochastic proposer, while a deterministic controller governs acceptance, retry, rollback, decomposition, and escalation using output-divergence signals over matched source-target executions. This positioning is especially important for data science and analytics pipelines, where partial translations can appear syntactically correct while silently drifting in schemas, values, distributions, or downstream metrics.

3. The DICE Framework

3.1. Framework definition

Let the source repository be R^{src} and the in-progress translated repository at iteration t be R_t^{tgt}. Let the repository graph be $G = (V, A)$, where each node $v \in V$ is a migration unit and each directed edge $(v_i, v_j) \in A$ denotes a syntactic, data-flow, schema-lineage, configuration, or interface dependency. In DICE-SASPy, nodes may be SAS macros, DATA steps, PROC blocks, include files, or utility modules, and the controller may group adjacent nodes into coarser chunks. Let X be a fixed validation workload of end-to-end runs, monitored boundary checkpoints, and interface checks derived from R^{src}. The goal is to construct R_*^{tgt} whose outputs match the required source outputs over X while minimizing unresolved SAS/Python boundaries and output divergence.

Table 1. Drift signals and controller responses in DICE-SASPy.

Drift signal	Controller response
Interface drift	Block accept, retry, or escalate
Schema drift	Split or retry
Value drift	Retry local repair
Distributional drift	Rollback or escalate
Downstream metric drift	Rollback or escalate

At iteration t, DICE selects a current chunk $c_t \subseteq V$ and builds its validator-scoped semantic closure $K(c_t)$, which extends c_t with the dependencies, schemas, schema lineage, macro variables, include files, expected inputs and outputs, and downstream consumers needed by the current validator.

Validation on workload X returns two control signals: a scalar output-divergence score E_t that summarizes source-target mismatch at monitored boundaries, and a typed validation signature e_t that records either direct failures, such as syntax or unresolved-symbol errors, or drift categories at monitored boundaries. The validator monitors five drift signals against R^{src}: interface drift on expected datasets, files, and reports; schema drift on types, row counts, and key columns; value drift on aligned values, aggregates, and missing-value behavior; distributional drift on quantiles and histograms; and downstream metric drift on task-level outputs such as AUC, RMSE, or forecast accuracy. Table 1 maps the drift component of e_t to the typical controller response, while non-drift failures are handled directly by the Deterministic Controller below.

Given the validation record, the controller chooses one of six actions: *accept*, *retry*, *split*, *rollback*, *bottom-up*, or *escalate*. **Convergence-controlled** means that repository structure, validation feedback, and attempt history determine which edits may be kept. Operationally, convergence means that accepted changes continue to satisfy hard output and interface checks while E_t does not increase on workload X. The formulation is intended to make progress toward output equivalence explicit and inspectable, not to provide a proof over all possible inputs.

3.2. Runtime architecture

The runtime loop is shown in Figure 1, with the four stages labeled in the left margin. The current DICE-SASPy implementation is available for anonymous review as a repository artifact (Anonymous, 2026).

Context Builder. DICE maintains the validator-scoped closure $K(c_t)$ for the current check. Across attempts, missing lineage or dependency drift expands the closure, boundary-spanning failures force redecomposition into smaller chunks, and adjacent stable regions can merge to reduce hybrid boundaries.

Bounded LLM Proposal. The LLM is a stochastic proposer, not an autonomous migrator. Given c_t, $K(c_t)$, and attempt history, it returns a patch proposal Δ_t with the contents shown in Figure 1. Bounding the edit radius makes validation artifacts reusable and gives rollback a well-defined target.

Mixed-State Validation. Each iteration executes source and target over aligned inputs in workload X at the monitored boundaries listed in Table 1, with SASPy running the SAS side and Python validators inspecting translated artifacts and downstream outputs. Rather than relying on generated unit tests, whose credibility is itself uncertain, DICE anchors validation in real input/output pairs and integrated tests already present in the repository. After the hard-gate check, the validator emits the divergence score E_t, the typed failure e_t, and supporting traces. Exact equality is used where possible and tolerance-bounded checks where exact parity is unstable.

Deterministic Controller. The controller maps the validation record to one of the actions shown in Figure 1. Local syntax or unresolved-symbol failures trigger another bounded repair pass. Schema or dependency failures trigger redecomposition or bottom-up conversion of supporting leaves. Rising or oscillating divergence triggers rollback to the last stable checkpoint. Repeated failure or unresolved business logic escalates to a reviewer. Top-down migration is the default, but bottom-up passes are used when untranslated dependencies dominate and stable leaves can be converted first. As a result, progress is visible as a sequence of accepted patches, recovered failures, and explicit escalation points rather than as a single opaque repository rewrite.

4. Discussion and Conclusion

Repository-level code migration should be treated as **convergence-controlled** search, where stochastic LLM proposals are governed by deterministic validation and explicit control over decomposition, acceptance, retry, rollback, and human escalation. In that setting, **DICE** makes migration inspectable: it exposes chunk boundaries, diagnostics, rollback points, convergence status, and escalation events while keeping semantic closure, bounded edit radius, and validator-driven actions explicit through mixed-runtime validation. **DICE-SASPy** is the current SAS→Python instantiation, and the accompanying artifact illustrates the controller workflow on prototype examples while benchmark results remain future work.

DICE is an engineering control layer for auditable repository migration, not a proof of output equivalence over all possible inputs. Its claim is empirical and limited to validation workload X, monitored boundaries, and task-specific tolerances. Future evaluation will test DICE-SASPy on public or contributed SAS→Python repositories under fixed model-call budgets against whole-file translation, fixed-block translate-and-retry, execution-feedback translation agents, and DICE ablations, reporting end-to-end output parity, monitored-boundary pass rate, final output divergence on X, iterations to convergence, rollback count, human interventions, and validator cost. Because public SAS/Python pairs remain scarce, contributed repositories with fixtures, expected outputs, and boundary metadata would materially strengthen the study. Future work should also test whether the controller transfers to other analytical migration settings, such as MATLAB→Python, SAS→R, and R→Python.

References

Anonymous. DICE: Dynamic inhomogeneous conversion engine. Anonymous repository, https://anonymous.4open.science/r/dice-engine-C033, 2026. Anonymous implementation repository artifact for the DICE-SASPy framework; accessed 2026-05-13.

Anthropic. The Code Modernization Playbook: Transforming Legacy Systems with AI. eBook / white paper, 2026. URL https://resources.anthropic.com/code-modernization-playbook. Accessed: 2026-05-13.

Bouzenia, I., Devanbu, P., and Pradel, M. RepairAgent: An autonomous, LLM-based agent for program repair. *arXiv preprint*, 2024.

Feng, J., Qin, Z., Gao, C., Wang, R., Wang, C., Ma, Y., and Xie, X. On the effectiveness of context compression for repository-level tasks: An empirical investigation. *arXiv preprint*, 2026.

Gehring, J., Zheng, K., Copet, J., et al. RLEF: Grounding code LLMs in execution feedback with reinforcement learning. *arXiv preprint*, 2024.

Ibrahimzada, A. R., Ke, K., Pawagi, M., Abid, M. S., Pan, R., Sinha, S., and Jabbarvand, R. AlphaTrans: A neuro-

symbolic compositional approach for repository-level code translation and validation. *arXiv preprint*, 2024.

Li, X.-Y., Du, Y.-L., and Li, M. Enhancing large language models in long code translation through instrumentation and program state alignment. *arXiv preprint*, 2025.

Pan, R. et al. Toward executable repository-level code generation via environment alignment. arXiv preprint, 2026.

SAS Institute Inc. SASPy: SAS scripting wrapper for Python. GitHub repository, https://github.com/sassoftware/saspy, 2024. Python interface to SAS systems for mixed-runtime validation and SAS execution.

Tao, Y., Qin, Y., and Liu, Y. Retrieval-augmented code generation: A survey with focus on repository-level approaches. arXiv preprint, 2025.

Wang, Y., Wang, Y., Wang, S., et al. RepoTransBench: A real-world benchmark for repository-level code translation. *arXiv preprint*, 2024.

Yang, J. et al. SWE-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 2024.

Yuan, Z., Chen, W., Wang, H., et al. TransAgent: Enhancing LLM-based code translation via fine-grained execution alignment. In *Proceedings of the International Symposium on Foundations of Software Engineering*, 2026.

Zhang, H., David, C., Wang, M., Paulsen, B., and Kroening, D. Scalable, validated code translation of entire projects using large language models. *Proceedings of the ACM on Programming Languages*, 2025.